

Dendê Eventos: Introdução a Persistência de Dados

Autor: Lucas Ferreira Lan
Cesar Filipe Gomes
Rodrigo Souza Guimarães

Agenda

O objetivo dessa apresentação é mostrar a evolução do Dendê Eventos com a persistência real em um banco de dados e o tratamento de exceções, além de apresentar um novo projeto utilizando o Spring boot framework e suas funcionalidades.

1. Persistência com JDBC:

Conexão e configuração do banco com o JDBC.

2. CrudRepository:

Implementação da interface CrudRepository com operações básicas.

4. RowMapper:

Conversão dos dados retornados em entidades Java.

5. Exceções customizadas:

Criação de mensagens customizadas e específicas para exceções.

6. Diagrama de sequência:

Diagrama de sequência da User: Story Desativar usuário.

7. Swagger:

Contextualização do Swagger e demonstração.

8. Annotations:

Explicação das principais annotations.

9. Gerenciamento de memória:

Explicação de como funciona o gerenciamento de memória no Spring boot.

Agenda

O objetivo dessa apresentação é mostrar a evolução do Dendê Eventos com a persistência real em um banco de dados e o tratamento de exceções, além de apresentar um novo projeto utilizando o Spring boot framework e suas funcionalidades.

10. Considerações finais:
Considerações finais.

11. Referências:
Referências.

Persistência com JDBC



- application.properties: Arquivo de configuração

```
datasource.url=jdbc:mysql://localhost:3306/dende_eventos?useSSL=false&serverTimezone=UTC
datasource.username=user
datasource.password=password
datasource.driver-class-name=com.mysql.cj.jdbc.Driver
datasource.hikari.maximum-pool-size=10
datasource.hikari.minimum-idle=2
datasource.hikari.connection-timeout=30000
```

Persistência com JDBC



- Classe ConfigProperties: Responsável por acessar as configurações do arquivo application.properties
- Annotation @Value do framework da dndê eventos faz a injeção dos valores definidos no application.properties

```
public class ConfigProperties {  
  
    @Value(key = "datasource.url")  
    private String dbUrl;  
  
    @Value(key = "datasource.username")  
    private String dbUsername;  
  
    @Value(key = "datasource.password")  
    private String dbPassword;  
  
    @Value(key = "datasource.driver-class-name")  
    private String dbDriverClassName;  
  
    @Value(key = "datasource.hikari.maximum-pool-size")  
    private Integer hikariMaximumPoolSize;  
  
    @Value(key = "datasource.hikari.minimum-idle")  
    private Integer hikariMinimumIdle;  
  
    @Value(key = "datasource.hikari.connection-timeout")  
    private Long hikariConnectionTimeout;  
  
}
```


Persistência com JDBC



- Classe `ConnectionPool`:
Responsável por gerenciar o pool das conexões com o banco.
- Reutilização de conexões com o banco de dados.
- Maior eficiência e gerenciamento de recursos.

```
public class ConnectionPool {  
    private final HikariDataSource dataSource;  
  
    public ConnectionPool(ConfigProperties configProperties){  
        HikariConfig hikariConfig = new HikariConfig();  
  
        hikariConfig.setJdbcUrl(configProperties.getDbUrl());  
        hikariConfig.setUsername(configProperties.getDbUsername());  
        hikariConfig.setPassword(configProperties.getDbPassword());  
        hikariConfig.setDriverClassName(configProperties.getDbDriverClassName());  
  
        hikariConfig.setMaximumPoolSize(configProperties.getHikariMaximumPoolSize());  
        hikariConfig.setMinimumIdle(configProperties.getHikariMinimumIdle());  
        hikariConfig.setConnectionTimeout(configProperties.getHikariConnectionTimeout());  
  
        this.dataSource = new HikariDataSource(hikariConfig);  
    }  
  
    public void close() {  
        if (this.dataSource != null) {  
            this.dataSource.close();  
        }  
    }  
}
```

CrudRepository



- Interface com operações como CREATE, READ, UPDATE e DELETE.
- Evita repetição de código.
- Reutilização de código e padronização.

CrudRepository



```
public class OrganizadorRepositorio implements CrudRepository<Organizador, Long>, RowMapper<Organizador>
{
    @Override
    public Optional<Organizador> findById(Long aLong) {
        String sql = "SELECT * FROM usuarios WHERE id = ?";

        try (Connection conn = ConnectionPool.getInstance().getDataSource().getConnection();
            PreparedStatement statement = conn.prepareStatement(sql)) {

            statement.setLong(1, aLong);

            try (ResultSet resultSet = statement.executeQuery()) {
                if (resultSet.next()) {
                    String[] row = new String[12];

                    row[0] = resultSet.getString("id");
                    row[1] = resultSet.getString("nome");
                    row[2] = resultSet.getString("data_nascimento");
                    row[3] = resultSet.getString("sexo");
                    row[4] = resultSet.getString("email");
                    row[5] = resultSet.getString("senha");
                    row[6] = resultSet.getString("is_empresa");
                    row[7] = resultSet.getString("cnpj");
                    row[8] = resultSet.getString("razao_social");
                    row[9] = resultSet.getString("nome_fantasia");
                    row[10] = resultSet.getString("is_ativo");
                    row[11] = resultSet.getString("has_evento");

                    return Optional.ofNullable(mapRow(row));
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }

        return Optional.empty();
    }
}
```


RowMapper



- Mapeamento e conversão dos dados retornados da consulta do banco de dados em objetos.
- Manipulação de dados, organização, reutilização e separação lógica de banco de dados e objetos da aplicação.

```
@Override
public Organizador mapRow(String[] row) {
    if (row == null) return null;

    Empresa empresa = null;
    if (Boolean.parseBoolean(row[6])) {
        empresa = Empresa.builder()
            .cnpj(row[7])
            .razaoSocial(row[8])
            .nomeFantasia(row[9])
            .build();
    }

    return Organizador.builder()
        .id(Long.parseLong(row[0]))
        .nome(row[1])
        .dataNascimento(LocalDate.parse(row[2]))
        .sexo(row[3])
        .email(row[4])
        .senha(row[5])
        .isEmpresa(Boolean.valueOf(row[6]))
        .empresa(empresa)
        .isAtivo(Boolean.valueOf(row[9]))
        .hasEvento(Boolean.valueOf(row[10]))
        .build();
    }
}
```

Exceções customizadas



- Exceções personalizadas para casos conhecidos de exceções.
- Manutenção e legibilidade.
- Exception e RuntimeException.

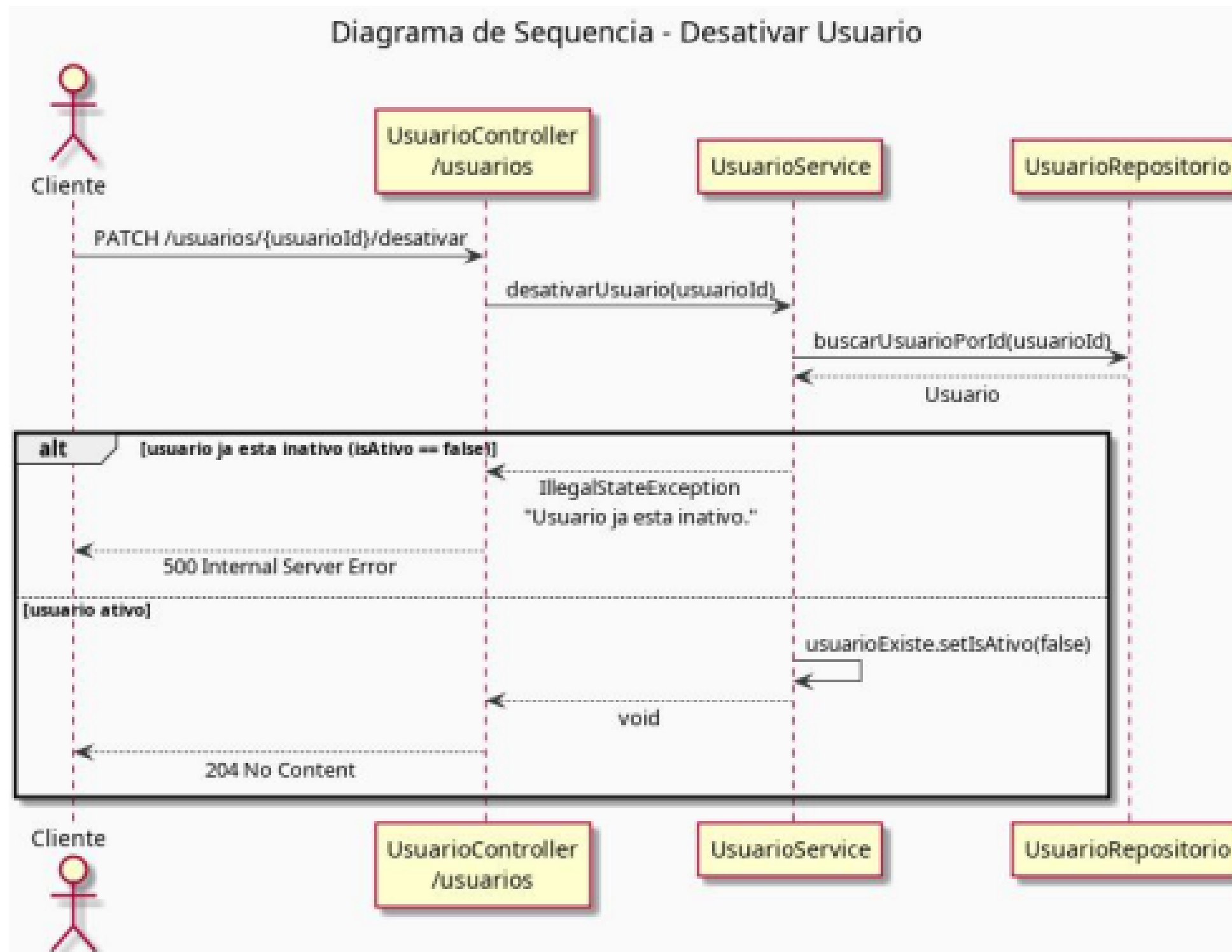
```
public class ConflictException extends RuntimeException {  
    public ConflictException(String mensagem) {  
        super(mensagem);  
    }  
}
```

Exceções customizadas



```
public Usuario criarUsuario(CriarUsuarioRequestDTO dto) {  
    this.usuarioRepositorio.findByField("email", dto.email()).ifPresent(usuario -> {  
        throw new ConflictException("O email já está em uso por outro usuário");  
    });  
  
    Usuario usuario = Usuario.builder()  
        .nome(dto.nome())  
        .email(dto.email())  
        .sexo(dto.sexo())  
        .senha(dto.senha())  
        .isAtivo(true)  
        .dataNascimento(dto.dataNascimento())  
        .build();  
  
    return this.usuarioRepositorio.save(usuario);  
}
```

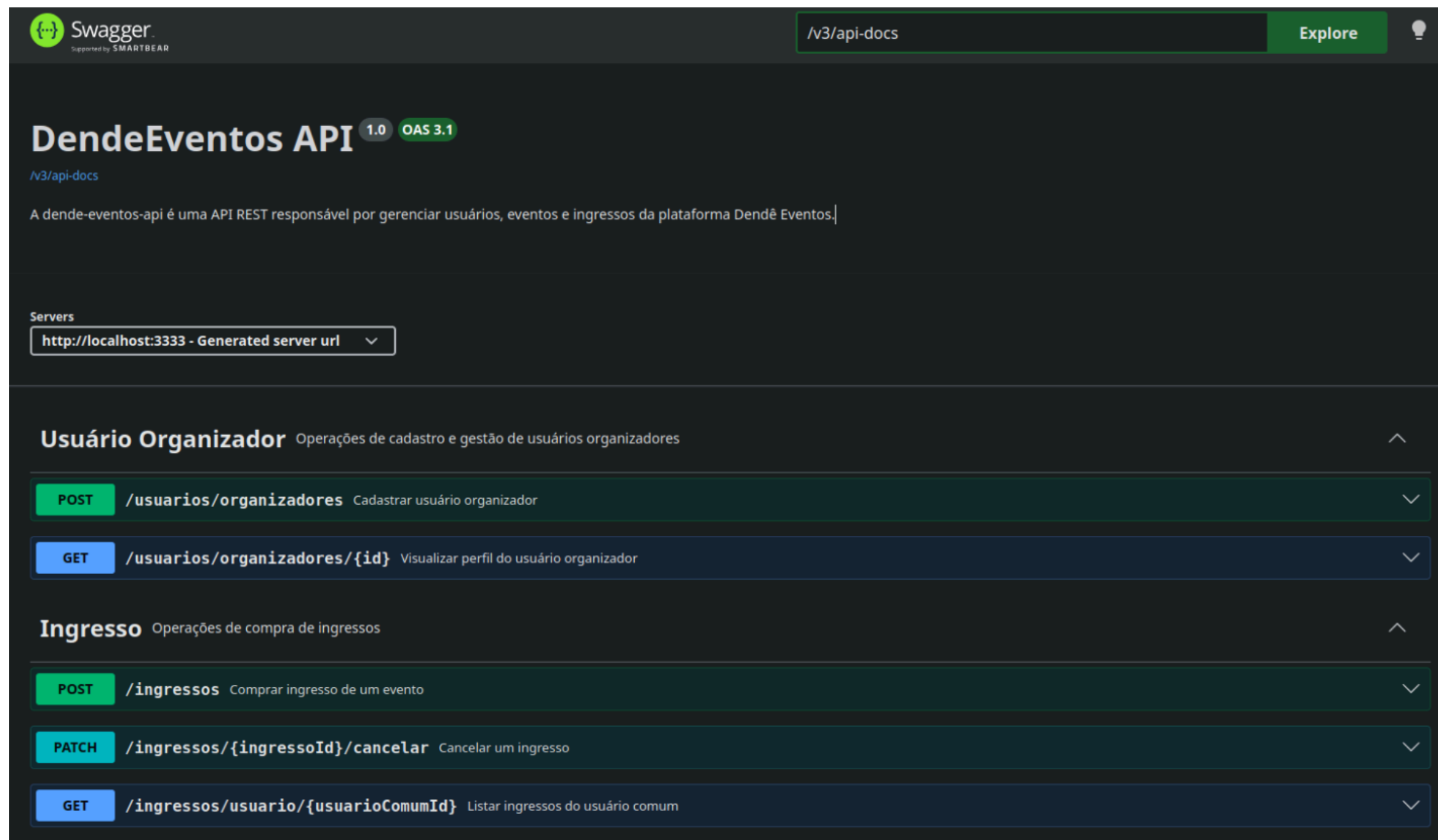
Diagrama de sequência



Swagger



- Documentação, visualização e teste
- Interface web interativa



Swagger



- Teste da rota

A screenshot of the Swagger UI interface. At the top, a blue button labeled 'GET' is next to the endpoint path '/usuarios/organizadores/{id}'. To the right of the path is the description 'Visualizar perfil do usuário organizador'. Below this, a 'Parameters' tab is selected, showing a table with two columns: 'Name' and 'Description'. A single parameter is listed: 'id', marked as 'required' with a red asterisk. Its type is 'integer(\$int64)' and its location is '(path)'. A text input field next to the parameter name contains the value '4'. At the bottom of the interface, there are two buttons: a blue 'Execute' button and a white 'Clear' button with a black border. A red-outlined 'Cancel' button is located in the top right corner of the parameters section.

GET /usuarios/organizadores/{id} Visualizar perfil do usuário organizador

Parameters

Name	Description
id * required integer(\$int64) (path)	4

Execute Clear

Cancel

Swagger



- Resposta da rota chamada

A screenshot of the Swagger UI interface. The 'Curl' section shows a GET request to 'http://localhost:3333/usuarios/organizadores/4' with an 'accept: */*' header. The 'Request URL' section shows the same URL. The 'Server response' section shows a 200 status code. The 'Response body' section displays a JSON object for a user named Cesar. The 'Response headers' section shows standard headers like 'content-type: application/json' and 'date: Thu, 04 Jun 2026 01:01:39 GMT'.

Curl

```
curl -X 'GET' \
'http://localhost:3333/usuarios/organizadores/4' \
-H 'accept: */*'
```

Request URL

http://localhost:3333/usuarios/organizadores/4

Server response

Code	Details
200	Response body <pre>{ "id": 4, "nome": "Cesar", "email": "cesar2@email.com", "dataNascimento": "2004-09-03", "sexo": "M", "idade": "21 anos, 9 meses e 0 dias", "isAtivo": true, "tipoUsuario": "ORGANIZADOR", "empresa": null }</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Thu, 04 Jun 2026 01:01:39 GMT keep-alive: timeout=60 transfer-encoding: chunked</pre>

Swagger



- Possíveis respostas

Responses

Code	Description	Links
200	Perfil encontrado	No links
Media type */ Controls Accept header. Example Value Schema <pre>{ "id": 0, "nome": "string", "email": "string", "dataNascimento": "2026-06-04", "sexo": "string", "idade": "string", "isAtivo": true, "tipoUsuario": "COMUM", "empresa": { "id": 0, "cnpj": "string", "razaoSocial": "string", "nomeFantasia": "string" } }</pre>		
404	Usuário não encontrado	No links
Media type */ Examples Usuário não encontrado Example Value Schema <pre>{ "status": 404, "mensagem": "Usuário não encontrado" }</pre> Example Description Usuário não encontrado		

Dendê Eventos: Introdução a Persistência de Dados

Annotations



- Metadados que fornecem informações de configuração e gerenciamento de componentes.
- @SpringBootApplication
- @Configuration,
@EnableAutoConfiguration,
@ComponentScan

```
package com.dendeframework.dendeeventos.londres;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LondresApplication {

    public static void main(String[] args) {
        SpringApplication.run(LondresApplication.class, args);
    }

}

return go(f, seed, [])
}
```

Annotations



- Sem o uso de Annotations

```
public class Servidor {  
  
    public static void main(String[] args) throws IOException {  
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);  
  
        server.createContext("/ola", new HttpHandler() {  
            @Override  
            public void handle(HttpExchange exchange) throws IOException {  
                String resposta = "Olá, Bem-vindo à nossa API!";  
  
                exchange.sendResponseHeaders(200, resposta.getBytes().length);  
  
                OutputStream os = exchange.getResponseBody();  
  
                os.write(resposta.getBytes());  
  
                os.close();  
            }  
        });  
  
        server.setExecutor(null);  
        server.start();  
        System.out.println("Servidor rodando na porta 8080...");  
    }  
}
```

Annotations



- Usando as annotations
@RestController e @GetMapping

```
@RestController
public class SaudacaoController {

    @GetMapping("/ola")
    public String dizerOla() {
        return "Olá, Bem-vindo à nossa API!";
    }
}
```

Gerenciamento de memória



- Memória heap: Young e Old Generation
- Bean: Objeto criado e gerenciado pelo Spring
- ApplicationContext: Cria, configura, armazena e destrói beans

Gerenciamento de memória



- Beans com escopo Singleton → Vivem até o contexto estar vivo. Exemplo: @Service
- Beans com escopo Prototype → Criados a cada injeção e coletados pelo Garbage Collector(GC).
- Escopos Request e Session → Criados e destruídos junto com a requisição.

Considerações finais



- Persistência real em banco de dados com JDBC.
- Otimização geral do sistema.
- Refatoração dos repositórios.
- Exceções personalizadas, garantindo respostas mais precisas.

Referências



- GEEKSFORGEEKS. **Simplifying CRUD Operation with JDBC.** GeeksforGeeks, 2023. Disponível em: <https://www.geeksforgeeks.org/java/simplifying-crud-operation-with-jdbc/>. Acesso em: 15 maio 2026.
- SPRING. **Spring Boot Documentation.** Spring, 2026. Disponível em: <https://docs.spring.io/spring-boot/documentation.html>. Acesso em: 30 maio. 2026.